



Stratax Protocol Audit Report

Version 1.0

SymmaTe

February 25, 2026

Protocol Audit Report

SymmaTe

February 25, 2026

Prepared by: SymmaTe Lead Security Researcher: - SymmaTe

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
- Medium
 - [M-1] `_executeOpenOperation` passes wrong token to `_call1InchSwap`, causing DoS or incorrect accounting
- Informational
- Gas

Protocol Summary

Stratax is a DeFi leveraged position protocol that enables users to create leveraged exposure to crypto assets using Aave V3 lending pools and 1inch DEX aggregator. The protocol uses flash loans to achieve capital-efficient leverage without requiring users to manually manage complex borrowing and swapping operations.

Key features: - Leveraged position creation using Aave V3 flash loans - Automated position unwinding with debt repayment - Integration with 1inch for optimal swap execution - Chainlink oracle integration for accurate price feeds

Disclaimer

The SymmaTe team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Contest: Cyfrin First Flight — Stratax Contracts **Contest Period:** February 12–19, 2026 **Platform:** CodeHawks

Scope

```
1 src/  
2 |-- Stratax.sol  
3 `-- StrataxOracle.sol
```

Roles

- **Position Owner:** Can create leveraged positions, unwind positions, supply/withdraw collateral, borrow debt tokens, and repay debt. Cannot make the position unhealthy (health factor < 1).
- **Oracle Owner:** Can set and update Chainlink price feeds for supported tokens (8 decimals precision only).

Executive Summary

Category	Details
Solidity Version	^0.8.13
Chain(s)	Ethereum Mainnet, EVM-compatible chains with Aave V3 + 1inch + Chainlink
nSLOC	356

Issues found

Severity	Number of issues found
High	0
Medium	1
Low	0
Informational	0
Gas	0
Total	1

Findings

High

None.

Medium

[M-1] `_executeOpenOperation` passes wrong token to `_call1InchSwap`, causing DoS or incorrect accounting

Description:

In `Stratax::_executeOpenOperation` (L513-514), `flashParams.borrowToken` is passed as the second parameter to `_call1InchSwap`, but the correct value should be `_asset` (the collateral token). This parameter is used in the fallback branch of `_call1InchSwap` (L625) to query the contract's token balance when the 1inch router returns no data:

```
1 // L513-514: borrowToken passed instead of collateralToken
2 uint256 returnAmount =
3     _call1InchSwap(flashParams.oneInchSwapData, flashParams.borrowToken
4         , flashParams.minReturnAmount);
```

```
1 // L623-625: fallback queries wrong token balance
2 } else {
3     returnAmount = IERC20(_asset).balanceOf(address(this));
4 }
```

Since all `borrowToken` was already swapped away at this point, the balance query returns ~0, causing `require(returnAmount >= totalDebt)` to revert.

Impact:

When the 1inch swap function returns no data, `createLeveragedPosition` always reverts, making leveraged position creation impossible through those swap routes. If the contract holds pre-existing `borrowToken`, the stale balance could bypass checks, leading to incorrect flash loan repayment accounting.

Proof of Concept:

```
1 function test_M1_wrongAssetCausesRevert() public {
2     // Setup: deploy Stratax with a mock 1inch router that swaps
3     // correctly but returns NO data
```

```
3      // Mock router does: pull WETH from caller, send USDC to recipient,
      return nothing
4
5      uint256 collateralAmount = 1000 * 1e6; // 1000 USDC
6      uint256 flashLoanAmount = 2000 * 1e6;
7      uint256 borrowAmount = 1 * 1e18;      // 1 WETH
8
9      usdc.mint(ownerTrader, collateralAmount);
10     bytes memory swapData = abi.encodeWithSelector(
11         Mock1inchRouterNoReturn.swap.selector, address(weth), address(
            usdc), borrowAmount, address(stratax));
12
13     vm.startPrank(ownerTrader);
14     usdc.approve(address(stratax), collateralAmount);
15
16     // Reverts: fallback queries WETH balance (0) instead of USDC
        balance (swap output)
17     vm.expectRevert();
18     stratax.createLeveragedPosition(
19         address(usdc), flashLoanAmount, collateralAmount, address(weth)
            , borrowAmount, swapData, flashLoanAmount);
20     vm.stopPrank();
21 }
```

Full runnable test: `test/audit/H1_WrongAssetInSwap.t.sol`

Recommended Mitigation:

Pass the collateral token address instead of `borrowToken`:

```
1      uint256 returnAmount =
2      -      _call1InchSwap(flashParams.oneInchSwapData, flashParams.
        borrowToken, flashParams.minReturnAmount);
3      +      _call1InchSwap(flashParams.oneInchSwapData, flashParams.
        collateralToken, flashParams.minReturnAmount);
```

Low

None.

Informational

None.

Gas

None.